Structs in Go

A **struct** is a **user-defined type that groups variables together**.

It is similar to a **class without inheritance**.

Syntax

```
type StructName struct {
    field1 datatype
    field2 datatype
}
```

Example

```
package main

import "fmt"

type Person struct {
    Name string
    Age  int
}

func main() {

    var p Person
```

```
    p.Name = "Dhivya"
    p.Age = 30

    fmt.Println(p.Name, p.Age)
}
```

Output

Dhivya 30

---

2. Struct Initialization

1️⃣ **Field by field**

```
var p Person
p.Name = "Dhivya"
p.Age = 30
```

2️⃣ **Using struct literal**

```
p := Person{
    Name: "Dhivya",
    Age:  30,
}
```

3️⃣ **Without field names**

Order must match struct definition.

```
p := Person{"Dhivya", 30}
```

---

3. Methods in Go

A **method is a function attached to a struct type**.

Syntax

```
func (receiver StructType) methodName() {
}
```

Example

```
package main

import "fmt"

type Person struct {
    Name string
    Age  int
}

func (p Person) Greet() {
    fmt.Println("Hello my name is", p.Name)
}

func main() {
```

```
    p := Person{"Dhivya", 30}

    p.Greet()
}
```

Output

Hello my name is Dhivya

---

4. Pointer Receiver vs Value Receiver

Value Receiver

Creates a **copy of struct**.

```
func (p Person) changeAge() {
    p.Age = 50
}
```

Original struct **NOT modified**.

Pointer Receiver

Modifies the **original struct**.

```
func (p *Person) changeAge() {
    p.Age = 50
}
```

Example

```
package main
```

```
import "fmt"

type Person struct {
    Name string
    Age  int
}

func (p *Person) changeAge() {
    p.Age = 40
}

func main() {

    p := Person{"Dhivya", 30}

    p.changeAge()

    fmt.Println(p.Age)
}
```

Output

40

---

5. Struct Embedding (Go Inheritance Alternative)

Go **does NOT support inheritance**.

Instead it uses **composition through embedding**.

Example

```
package main

import "fmt"

type Address struct {
    City  string
    State string
}

type Person struct {
    Name string
    Age  int
    Address
}

func main() {

    p := Person{
```

```
        Name: "Dhivya",
        Age:  30,
        Address: Address{
            City:  "Chennai",
            State: "Tamil Nadu",
        },
    }

    fmt.Println(p.Name)
    fmt.Println(p.City)
}
```

Output

Dhivya

Chennai

Notice:

p.Address.City

p.City ← promoted field

---

6. Pointers with Struct

You can create **pointer to struct**.

Example

```
p := &Person{
```

```
    Name: "Dhivya",
    Age:  30,
}

fmt.Println(p.Name)
```

Go automatically **dereferences pointer**.

Equivalent to

```
(*p).Name
```

---

7. Passing Struct to Functions

Pass by Value

Copy is created.

```
func updateAge(p Person) {
    p.Age = 40
}
```

Original **not changed**.

Pass by Pointer

```
func updateAge(p *Person) {
    p.Age = 40
}
```

Example

```
package main
```

```
import "fmt"

type Person struct {
    Name string
    Age  int
}

func updateAge(p *Person) {
    p.Age = 40
}

func main() {

    p := Person{"Dhivya", 30}

    updateAge(&p)

    fmt.Println(p.Age)
}
```

Output

40

---

8. Returning Struct from Function

Example

```
func createPerson(name string, age int) Person {

    p := Person{
        Name: name,
        Age:  age,
    }

    return p
}
```

Usage

```
func main() {

    p := createPerson("Dhivya", 30)

    fmt.Println(p)
}
```

---

9. Returning Pointer to Struct

More efficient for large structs.

```
func createPerson(name string, age int) *Person {
```

```
    return &Person{
        Name: name,
        Age:  age,
    }
}
```

Usage

```
p := createPerson("Dhivya", 30)

fmt.Println(p.Name)
```

---

10. Complete Example (Everything Together)

```
package main

import "fmt"

type Address struct {
    City string
}

type Person struct {
    Name string
    Age  int
    Address
```

```
}

func (p *Person) updateAge(age int) {
    p.Age = age
}

func newPerson(name string, age int, city string) *Person {

    return &Person{
        Name: name,
        Age:  age,
        Address: Address{
            City: city,
        },
    }
}

func main() {

    p := newPerson("Dhivya", 30, "Chennai")

    p.updateAge(35)
```

```
    fmt.Println(p.Name)
    fmt.Println(p.Age)
    fmt.Println(p.City)
}
```

Output

```
Dhivya
35
Chennai
```

---

11. Concept Map

```
Struct
    │
    ├── Fields
    │
    ├── Methods
    │       ├── Value Receiver
    │       └──── Pointer Receiver
    │
    ├── Embedding (Composition)
    │
    ├── Struct Pointer
    │
    ├── Pass to Function
```

```
│       ├── Value
│       └──── Pointer
│
└──── Return from Function
    ├── Struct
    └──── Pointer
```

Anonymous structs

In **Go**, an **Anonymous Struct** is a struct **without a named type**.
You define and use it **directly at the point where it is needed**.

It is useful when:

- You need a **temporary struct**
- You don't want to create a **separate type**
- Used heavily in **JSON responses, testing, and quick data grouping**

---

1. Basic Anonymous Struct

Syntax

```
var variableName struct {
   field1 type
   field2 type
}
```

Example

```
package main
```

```go
import "fmt"

func main() {

    var person struct {
        Name string
        Age  int
    }

    person.Name = "Dhivya"
    person.Age = 30

    fmt.Println(person)
}
```

Output

{Dhivya 30}

Here the struct **does not have a type name**.

---

2. Anonymous Struct with Initialization

More common usage.

```go
package main
```

```
import "fmt"

func main() {

    person := struct {
        Name string
        Age  int
    }{
        Name: "Dhivya",
        Age:  30,
    }

    fmt.Println(person.Name)
}
```

Output

Dhivya

Structure:

struct definition

+

struct value

struct { fields } { values }

---

3. Anonymous Struct Inside Slice

Often used when returning **API responses**.

```
package main

import "fmt"

func main() {

    users := []struct {
        Name string
        Age  int
    }{
        {"Dhivya", 30},
        {"Shirisha", 10},
    }

    for _, user := range users {
        fmt.Println(user.Name, user.Age)
    }
}
```

Output

```
Dhivya 30
Shirisha 10
```

---

4. Anonymous Struct as Function Return

```
package main

import "fmt"

func getUser() struct {
    Name string
    Age  int
} {

    return struct {
        Name string
        Age  int
    }{
        Name: "Dhivya",
        Age:  30,
    }
}

func main() {

    user := getUser()
```

```
    fmt.Println(user.Name)
}
```

---

5. Anonymous Struct in Map

```
package main

import "fmt"

func main() {

    employees := map[int]struct {
        Name string
        Age  int
    }{
        1: {"Dhivya", 30},
        2: {"Arun", 28},
    }

    fmt.Println(employees[1].Name)
}
```

---

6. Anonymous Struct with JSON (Very Common)

Used when creating **quick API responses**.

```go
package main

import (
    "encoding/json"
    "fmt"
)

func main() {

    response := struct {
        Status  string
        Message string
    }{
        Status:  "success",
        Message: "User created",
    }

    jsonData, _ := json.Marshal(response)

    fmt.Println(string(jsonData))
}
```

Output

```
{"Status":"success","Message":"User created"}
```

---

7. Anonymous Struct with Pointer

```
package main

import "fmt"

func main() {

    p := &struct {
        Name string
        Age  int
    }{
        Name: "Dhivya",
        Age:  30,
    }

    fmt.Println(p.Name)
}
```

---

8. Anonymous Struct as Embedded Field

```
package main

import "fmt"
```

```go
type Employee struct {
    Name string

    struct {
        City  string
        State string
    }
}

func main() {

    e := Employee{
        Name: "Dhivya",
    }

    e.City = "Chennai"

    fmt.Println(e.City)
}
```

---

9. When Anonymous Structs Are Used

Common places in **real Go applications**.

1️⃣ **JSON responses**

API response objects

2️⃣ **Testing**

table-driven tests

3️⃣ **Quick grouping of data**

temporary data structure

4️⃣ **Channels**

chan struct{}

5️⃣ **Map values**

map[string]struct{}

---

10. Difference

Feature	Named Struct	Anonymous Struct
Has type name	Yes	No
Reusable	Yes	No
Memory allocation	Same	Same
Use case	Models, entities	Temporary structures

Example

Named struct

```
type Person struct {
    Name string
    Age int
```

```
}
```

Anonymous struct

```
struct {
    Name string
    Age int
}
```

---

11. Important Real Go Pattern (Zero Memory Struct)

Very common in Go.

```
set := map[string]struct{}{}

set["apple"] = struct{}{}
set["banana"] = struct{}{}
```

Used to implement **Set data structure**.

Because:

struct{} takes ZERO bytes of memory

---

12. Visual Understanding

Named Struct

```
type Person struct {
   Name string
   Age int
```

```
}
```

Anonymous Struct

```
struct {
  Name string
  Age int
}
```

✅ Since you are progressing well in **Go learning**, the **next critical topic** after this is:

Struct Tags (VERY important for backend)

Example:

```
type User struct {
   Name string `json:"name"`
   Age  int    `json:"age"`
}
```

Used for:

- JSON
- Database ORM
- API serialization

These are **very important for backend development and microservices in Go**.

Constructor pattern in Go

Go does **not have constructors like Java or C++**, but Go developers commonly use a **constructor pattern** using a **function that initializes and returns a struct**.

Usually the function name starts with **New**.

Example:

NewUser()

NewServer()

NewDatabase()

NewPerson()

This is a **Go convention**.

---

1. Basic Constructor Pattern

Struct

```
type Person struct {
    Name string
    Age  int
}
```

Constructor Function

```
func NewPerson(name string, age int) Person {
    return Person{
            Name: name,
            Age:  age,
    }
}
```

Usage

```
package main

import "fmt"

type Person struct {
    Name string
    Age  int
}

func NewPerson(name string, age int) Person {
    return Person{
            Name: name,
            Age:  age,
    }
}

func main() {

    p := NewPerson("Dhivya", 30)

    fmt.Println(p)
}
```

Output

{Dhivya 30}

---

2. Constructor Returning Pointer (Most Common)

Most Go constructors return **pointer to struct**.

Why?

- Avoid copying large structs
- Methods often use pointer receivers
- Efficient memory usage

Example

```
package main

import "fmt"

type Person struct {
    Name string
    Age  int
}

func NewPerson(name string, age int) *Person {
    return &Person{
            Name: name,
            Age:  age,
```

```
    }
}

func main() {

    p := NewPerson("Dhivya", 30)

    fmt.Println(p.Name)
}
```

---

3. Constructor with Default Values

Constructors can **initialize default values**.

```
type Server struct {
    Host string
    Port int
}
```

Constructor

```
func NewServer() *Server {

    return &Server{
            Host: "localhost",
            Port: 8080,
    }
```

```
}
```

Usage

```
s := NewServer()

fmt.Println(s.Host, s.Port)
```

Output

```
localhost 8080
```

---

4. Constructor with Validation

Constructors can enforce **rules or validation**.

Example:

```
type User struct {
    Name string
    Age  int
}
```

Constructor

```
func NewUser(name string, age int) (*User, error) {

    if age < 0 {
            return nil, fmt.Errorf("age cannot be negative")
    }

    return &User{
```

```
            Name: name,
            Age:  age,
    }, nil
}
```

Usage

```
user, err := NewUser("Dhivya", 30)

if err != nil {
    fmt.Println(err)
    return
}

fmt.Println(user.Name)
```

---

5. Constructor with Private Fields

Constructor pattern is also used to **protect struct fields**.

Example

```
type Account struct {
    balance float64
}
```

Constructor

```
func NewAccount(balance float64) *Account {
    return &Account{
```

```
            balance: balance,
    }
}
```

Methods

```
func (a *Account) Deposit(amount float64) {
    a.balance += amount
}

func (a *Account) Balance() float64 {
    return a.balance
}
```

Usage

```
acc := NewAccount(1000)

acc.Deposit(500)

fmt.Println(acc.Balance())
```

---

6. Constructor with Embedded Structs

```
type Address struct {
    City string
}
```

```
type Person struct {
    Name string
    Address
}
```

Constructor

```
func NewPerson(name string, city string) *Person {

    return &Person{
            Name: name,
            Address: Address{
                    City: city,
            },
    }
}
```

Usage

```
p := NewPerson("Dhivya", "Chennai")

fmt.Println(p.City)
```

---

7. Multiple Constructors (Factory Style)

Sometimes multiple constructors are used.

Example

```
func NewAdmin(name string) *User {
```

```
    return &User{
            Name: name,
            Role: "admin",
    }
}

func NewGuest() *User {
    return &User{
            Name: "guest",
            Role: "guest",
    }
}
```

---

8. Real World Example (HTTP Server)

Example similar to Go libraries.

```
type Server struct {
    host string
    port int
}
```

Constructor

```
func NewServer(host string, port int) *Server {
    return &Server{
            host: host,
```

```
        port: port,
    }
}
```

Method

```
func (s *Server) Start() {
    fmt.Println("Server started on", s.host, ":", s.port)
}
```

Usage

```
server := NewServer("localhost", 8080)

server.Start()
```

---

9. Why Go Uses Constructor Functions

Reason	Explanation
No classes	Go is not OOP like Java
Simpler design	Just functions
Validation possible	Constructors can check data
Encapsulation	Hide private fields
Idiomatic Go	Standard pattern

---

10. Visual Comparison

Java

```
class Person {
```

```
    Person(String name) { }
}
```

Go

```
func NewPerson(name string) *Person
```

---

11. Important Go Convention

Constructor name:

New<TypeName>

Examples from Go standard library:

http.NewRequest()

bytes.NewBuffer()

errors.New()

---

12. Best Practice Summary

✓ Use New<Type> naming
✓ Return pointer for large structs
✓ Add validation if needed
✓ Initialize default values
✓ Hide internal fields when necessary